

Plano Xclipse Open 940 — XO940

Abordagem 5 dois meios e plano técnico baseado em evidências

Autor: Manus AI

Revisão: 05 de setembro de 2026

Alvo: Samsung SM-S721B / Xclipse 940

Sumário

1	Sumário executivo	1
2	1. Escopo e regra de evidência	1
3	2. Estado confirmado no pacote	1
4	3. Abordagem 5 dois meios	2
	4.1 3.1 Cinco descobertas essenciais	2
	4.2 3.2 Cinco descobertas interessantes	3
5	4. Ordem de execução e gates	4
6	5. Critérios de teste e encerramento	4
7	6. Entregáveis públicos do projeto	5
8	7. Referências	5
9	8. Atualização dos resultados obtidos	5
	9.1 8.1 Loader e caminho de produção	6
	9.2 8.2 Caminho SGPU e DRM	6
	9.3 8.3 Caminho real de compute	6
	9.4 8.4 Compiler, encoding e ISA	6
	9.5 8.5 Estado revisado dos cinco itens	6

1 Sumário executivo

Este plano atualiza o documento original do projeto **Xclipse Open 940** com o estado observado no pacote compartilhado e organiza a investigação pelo **abordagem 5 dois meios**: cinco descobertas essenciais para que o projeto possa avançar com segurança e cinco descobertas secundárias que podem aumentar a qualidade do driver, da documentação e da portabilidade.

O alvo interno do projeto é **XO940**. O primeiro aparelho de laboratório é o **Samsung SM-S721B**, identificado nos artefatos como r12s, plataforma erd9945 e hardware s5e9945. O objetivo imediato não é declarar um driver funcional. É produzir uma especificação verificável do contrato entre aplicação, loader Android, ICD vendor, driver de usuário, DRM/UAPI, firmware e GPU.

> **Conclusão atual**: já existe um caminho SGPU observável por DRM para identificação, consultas, alocação GTT, mapeamento de CPU e mapeamento de VA. Ainda não existe evidência de submissão de comandos, dispatch compute, readback produzido pela GPU, ISA decodificada, backend de compilador ou driver Vulkan independente.

2 1. Escopo e regra de evidência

O projeto combina documentação de GPU, reverse engineering controlado, bring-up de DRM, investigação de loader Android, desenvolvimento futuro de ferramentas e, somente quando a evidência justificar, um driver Vulkan/Mesa experimental para Xclipse. “Turnip-like” descreve apenas a categoria de um driver Vulkan aberto para Xclipse; não implica compatibilidade ou reutilização de código do Turnip. O mesmo vale para RADV.

A regra central é separar **inicialização, observação, capability query, construção de pipeline, submissão, execução, sincronização e validação de resultado**. Um arquivo chamado probe, test, exec ou diagnostico não é um teste de capacidade por si só.

Artefato	Pode estabelecer	Não estabelece sozinho
Probe SGPU	Consultas DRM, memória e VA executadas	Que a GPU recebeu ou concluiu trabalho
Pipeline Vulkan	Que objetos de pipeline foram aceitos pelo caminho visível	Dispatch, submit, execução ou readback
Mapeamento de BO	Visibilidade e mapeamento de uma alocação	Acesso correto da GPU ou coerência de cache
Queue submit	Que uma API aceitou uma submissão	Que firmware executou o IB ou produziu resultado
Readback esperado	Uma operação fim a fim passou	Compatibilidade geral entre revisões e workloads

3 2. Estado confirmado no pacote

O pacote Quick Share foi preservado fora do Git e recebeu o hash SHA-256 cd73e2fa24ac083b9babf243b39772065b060abc9cd30fb476acabe74433e67f. A fonte Samsung foi inventariada sem redistribuir o código no repositório. O header kernel/include/uapi/drm/sgpu_drm.h, o

módulo SGPU AMDGPU-derived, os Device Trees s5e9945, caminhos de IOMMU/DMA-BUF e referências de firmware foram localizados.

O relatório de dispositivo identifica o render node SGPU /dev/dri/renderD128, enquanto /dev/dri/renderD129 pertence ao caminho exynos-drm. A consulta de hardware informa família 147 (MGFX), device ID 0x000073a0, revisão 0x02600200, GFX IP 10.0 com máscara de rings 0xf e COMPUTE IP 10.0 com máscara 0x7. O firmware observado inclui SGPU 2.23.0 e RTL 0x4ea15.

O probe SGPU criou um BO GTT de 64 KiB, realizou mapeamento pela CPU, tocou a memória, mapeou VA, desmapeou VA e fechou o objeto. O código declara explicitamente que **não submete nada à GPU**. O probe Vulkan expõe apenas llvmpipe no namespace Termux; a criação de pipeline não alcançou uma execução Samsung.

4 3. Abordagem 5 dois meios

O abordagem não significa escolher cinco testes arbitrários. Cada item precisa conter uma hipótese, uma forma de execução, um resultado mínimo, uma evidência bruta e uma regra de promoção. Os cinco primeiros itens bloqueiam decisões arquiteturais. Os cinco seguintes são valiosos, mas não devem atrasar a prova dos fundamentos.

4.1 3.1 Cinco descobertas essenciais

N	Descoberta crítica	Teste mínimo e evidência	Critério de sucesso
1	Loader Android e VkPhysicalDevice Samsung	Mapear o processo Android suportado, namespace, manifest/HAL e ABI. Capturar o loader, vendor ID, device ID e caminho do ICD sem depender de dlopen em Termux.	Um processo suportado enumera o dispositivo Samsung 0x144d e o relatório preserva comando, namespace, build e saída bruta.
2	Contrato DRM, BO, VA, IOMMU e cache	Repetir consultas e alocações reversíveis no renderD128; registrar alinhamento, flags, VA, DMA-BUF, permissões e falhas.	A equipe descreve o caminho de memória e suas limitações sem afirmar acesso da GPU antes de uma validação de execução.
3	Contexto, ring, IB, fence e recuperação	Somente após documentar firmware e reset, identificar criação de contexto, BO list, chunk, ring, syncobj/fence e o caminho de erro.	Existe um experimento bounded que pode ser interrompido, sincronizado e recuperado sem deixar o aparelho em estado incerto.

4	Primeiro dispatch compute com readback	Submeter uma operação mínima e determinística, aguardar fence e comparar um valor produzido pela GPU com um valor esperado.	Há evidência do mesmo BO antes/depois, submit, conclusão, readback e recuperação; criação de pipeline não conta.
5	Correlação shader-binário-ISA	Usar pares de shaders controlados, obter binários ou metadados Samsung pelo caminho real e registrar hashes, tamanho e diferenças.	Ao menos uma forma de instrução ou campo é decodificada com confiança e reproduzida em mais de um caso controlado.

4.2 3.2 Cinco descobertas interessantes

N	Descoberta útil	Como investigar	Utilidade posterior
1	Preempção, doorbells e reset fino	Correlacionar scheduler, rings, doorbells, timeout, page fault e reset no código e nos logs; sem replay opaco.	Aumenta a robustez do driver e explica falhas que não aparecem em testes curtos.
2	Sequência de firmware e variação de revisão	Registrar blobs referenciados, versões, ordem de boot e diferenças entre revisões s5e9945, sempre como metadado ou referência licenciada.	Permite separar comportamento do XO940 de comportamento específico do firmware.
3	Power, clocks, OPP e thermal	Observar sysfs, Device Tree, thermal zones e estados de clock durante probes seguros e repetíveis.	Ajuda a explicar variação de desempenho, timeouts e limites térmicos.
4	Metadados de pipeline e extensões Vulkan	Quando o ICD real estiver visível, comparar VK_KHR_pipeline_creation_capabilities, cache, formatos, indexing, pipeline creation capabilities, pipeline creation	Pode acelerar a documentação

		barreiras e propriedades reportadas.	confundir metadados com ISA.
5	Generalização para outras revisões e Mesa	Manter tabelas por revisão, separar o contrato comum dos quirks e testar uma camada mínima antes de integração Mesa.	Evita que um resultado do SM-S721B seja promovido indevidamente para toda a família Xclipse.

5 4. Ordem de execução e gates

A ordem é deliberadamente conservadora. Primeiro vêm identidade, loader e plataforma. Depois memória e VM. Em seguida vêm filas, sincronização e recuperação. Só então compute, ISA, compiler, Vulkan e driver.

Fase	Objetivo	Gate de saída
0	Governança, hash, inventário, licença e perguntas abertas	Manifesto, inventários e STATUS versionados
1	Plataforma, DRM, UAPI, DT, firmware e loader	Mapa reproduzível do caminho até a GPU visível
2	Observabilidade segura e probes read-only	Saídas brutas, comandos e limites preservados
3	Memória, VA, IOMMU, cache e faults	Alocação e mapeamento reproduzíveis
4	Queues, command processor, fences e reset	Hipótese de packet/ring apoiada por evidência
5	Compute mínimo	Submit, sincronização e readback esperado
6	ISA e compiler	Uma forma de instrução correlacionada com confiança
7–10	Vulkan, layers, driver e manutenção	Feature Vulkan real, layer observável e regressões versionadas

6 5. Critérios de teste e encerramento

Um teste real deve indicar aparelho, build, revisão, firmware, data, comando exato, saída bruta, interpretação e nível de confiança. O relatório deve declarar qual camada foi atingida. O projeto deve registrar também abordagens descartadas, como o dlopen direto no namespace Termux, sem transformar essa falha específica em uma afirmação universal.

Nenhum teste de submissão deve usar command buffers opacos, escrever registradores sem necessidade documentada, alterar permanentemente SELinux ou modificar /system e /vendor. Cada experimento

precisa ter pré-condição, timeout, condição de abortar, recuperação e evidência de que o aparelho voltou ao estado esperado.

Rótulo	Uso	Exemplo XO940
Confirmado	Observado diretamente ou sustentado por fonte pública clara	renderD128 ligado ao SGPU; BO/VA probe concluído
Confirmado por fonte	Presente no kernel, documentação ou símbolo verificável	sgpu_drm.h, Device Tree e caminhos SGPU
Provável	Inferido de múltiplas evidências	Compatibilidade AMDGPU-derived além do subset observado
Hipótese	Possibilidade que exige experimento	Primeiro submit seguro em ring COMPUTE
Descartado	A abordagem específica falhou	Abrir ICD diretamente no namespace Termux

7 6. Entregáveis públicos do projeto

O repositório deve manter especificações por camada, inventários CSV, relatórios de experimentos, dados sanitizados do SM-S721B, classificações de probes e testes, e uma árvore que diferencie o que já existe do que está planejado. Código Samsung, firmware, bibliotecas vendor e dumps crus não são publicados automaticamente. A licença precisa ser avaliada por arquivo e a proveniência precisa acompanhar qualquer reutilização.

Este plano é um guia de trabalho, não um certificado de compatibilidade. O resultado bem-sucedido pode ser uma documentação suficiente para permitir decisões técnicas corretas, mesmo que alguma camada — loader, firmware, ISA, compiler ou reset — continue bloqueando um driver independente.

8 7. Referências

- [1] Arquivo Quick Share fornecido para o projeto XO940, com hash registrado em data/source-archive-manifest.txt.
- [2] reports/initial-evidence.md, relatório de evidências iniciais do pacote.
- [3] source-analysis/kernel-source-review.md, inventário curado do código Samsung.
- [4] docs/00-project-scope.md, escopo e regra de evidência do repositório.
- [5] ROADMAP.md, fases e critérios de saída do projeto.

9 8. Atualização dos resultados obtidos

O novo pacote 5doismeiosresultadosobtidos.zip foi analisado em 05 de setembro de 2026. Seu SHA-256 é 59ac6221373572024c0967d642f9ead1f970c6791ff2428cb5c49ae73a5fe7fc. O binário vendor foi mantido fora do repositório; somente metadados, caminhos e conclusões derivadas foram publicados.

9.1 8.1 Loader e caminho de produção

A coleta válida do SurfaceFlinger mostrou `/vendor/lib64/hw/vulkan.samsung.so` e `/system/lib64/libvulkan.so` mapeadas no processo, além de FDs apontando para `/dev/dri/renderD128`. O mount namespace do SurfaceFlinger era diferente do namespace Termux. Portanto, o resultado `llvmpipe` no Termux não invalida a presença do ICD Samsung no caminho de produção. A enumeração detalhada de `VkPhysicalDevice` nesse processo ainda não foi capturada.

9.2 8.2 Caminho SGPU e DRM

O pacote fecha o mapa `card0 → sgpu`, `card1 → exynos-drm`, `/dev/dri/renderD128 → SGPU` e `/dev/dri/renderD129 → Exynos/display`. A identidade `/sys/devices/platform/22200000.sgpu`, o compatível `samsung-sgpu`, `samsung-sgpu`, `firmware SGPU 2.23.0`, `RTL 0x0004ea15` e frequências `devfreq` foram correlacionados. A biblioteca `libdrm_sgpu.so` expõe, no inventário, interfaces de BO, VA, contexto, submit, fences e queries; isso mapeia capacidade, não prova de chamada executada.

9.3 8.3 Caminho real de compute

O processo correto do Photo Remaster foi identificado como `com.samsung.android.photoremasterservice:photoremasterservice`. Ele carregou `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so` e `libSGPUOpenCL.so`, mantendo FDs em `renderD128` e `card0`. A API OpenCL inclui contexto, fila, kernel, `clEnqueueNDRangeKernel`, sincronização e `clEnqueueReadBuffer`. A atividade do processo correlacionou-se com `runtime_status=active` e variação de frequência SGPU. Ainda não há a cadeia comprovada `B0 antes → submit → execução → fence/evento → readback → recuperação`.

9.4 8.4 Compiler, encoding e ISA

A análise estática do ICD stripped encontrou vocabulário de SPIR-V, shader compiler, `GetOpcode`, `XlateOpcode`, `GetHwOpcode`, `SCEmitter`, `SCAsmEncoder`, `VOPC/VOP1/VOP2/VOP3/VOP3P`, `DPP8/DPP16` e mensagens de encoding. Uma disassembly AArch64 real foi obtida usando o endereço virtual correto da seção `.text, 0x143fb30`. A evidência é forte para infraestrutura interna de compiler/encoding, mas strings isoladas e referências que terminam em logging não são símbolos de função. `gfx10_4_GEN` também não prova que XO940 seja GFX10.4. Ainda falta correlacionar shader controlado, binary gerado e ISA nativa em runtime.

9.5 8.5 Estado revisado dos cinco itens

Item	Estado	Conclusão correta
Loader Android	Parcialmente concluído	ICD e DRM observados no SurfaceFlinger; enumeração Vulkan detalhada ainda aberta.
DRM/BO/VA/IOMMU	Fortalecido	Estrutura SGPU, firmware, <code>libdrm</code> e interfaces mapeadas; chamadas individuais ainda precisam de correlação.

Contexto/ring/IB/fence	Mapeado	APIs de submit e sync identificadas; submit do projeto não demonstrado.
Dispatch/readback	Caminho real identificado	Photo Remaster/OpenCL/SGPU correlacionados; kernel próprio com readback não provado.
Shader/binário/ISA	Evidência estática forte	Compiler/encoding interno sustentado; ISA nativa em runtime ainda aberta.

O relatório operacional completo está em `reports/5-dois-meios-essenciais.md`, e a análise do pacote está em `reports/new-results-analysis.md`.